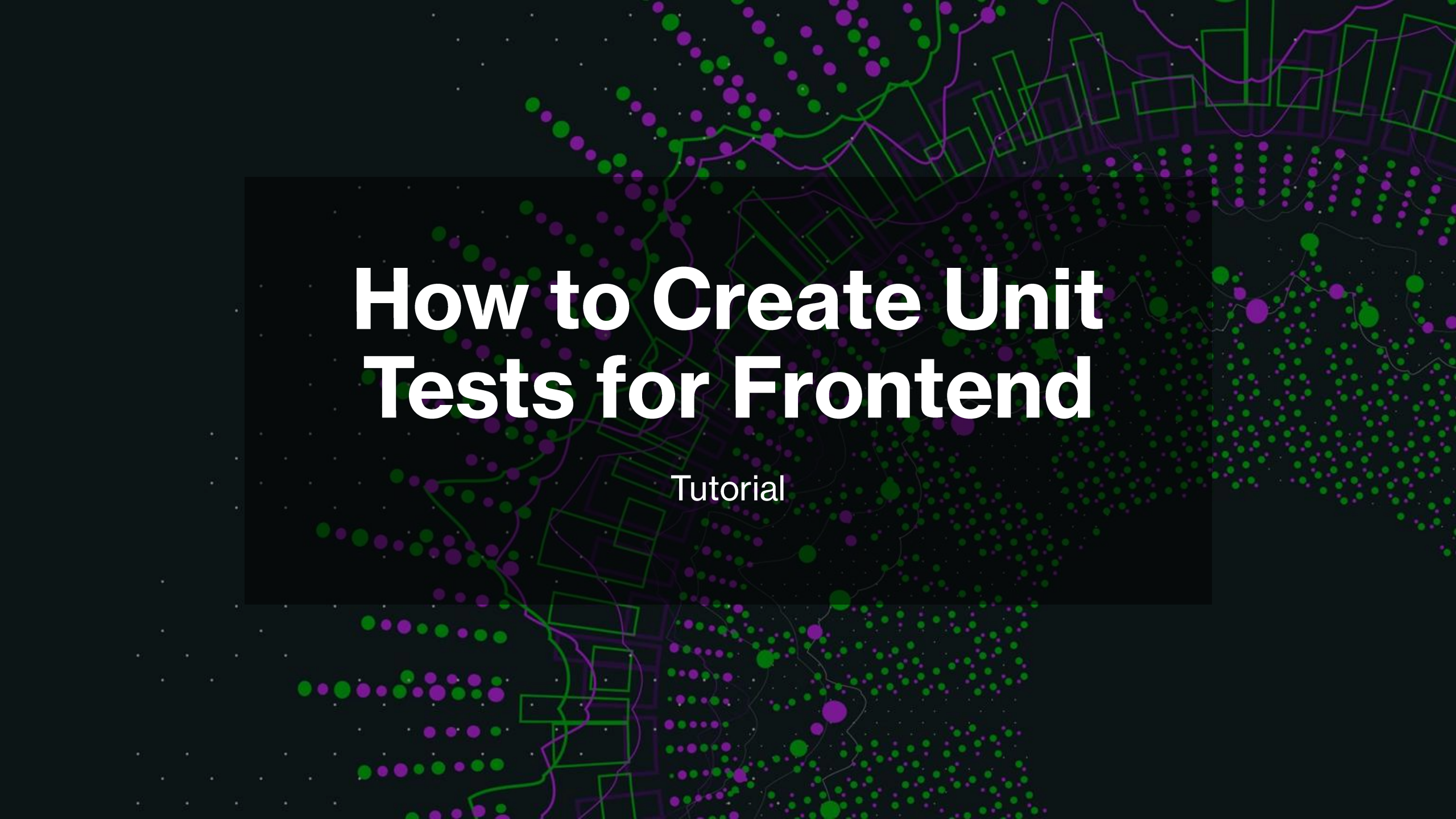




How to Create Unit Tests for Frontend

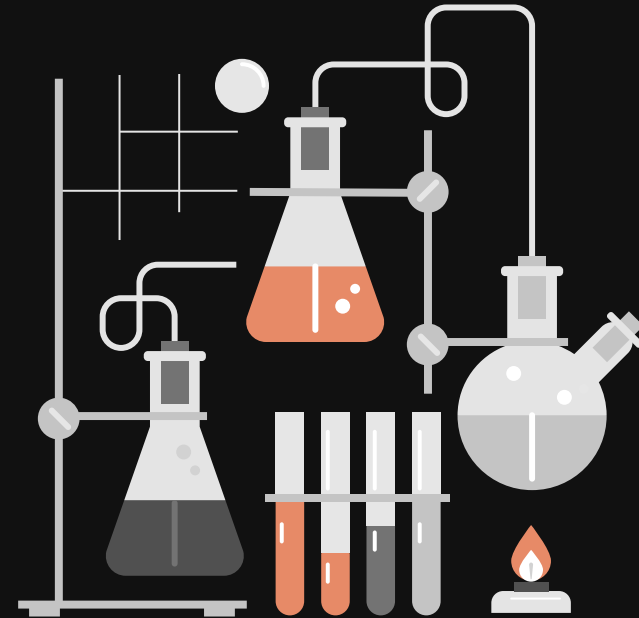
Tutorial

Frontend Requirements

- Input Handling
 - Buttons for hexadecimal digits
 - 0 to F
 - Buttons for arithmetic operators
 - An area to display what the user inputs
 - Clear Button
 - Backspace Button
- Arithmetic Calculations
 - Button for equal sign
- Output Handling
 - An area to display what the output of a calculation
- Error Handling
 - A toast to display errors
 - Validation for input
 - Validation for output

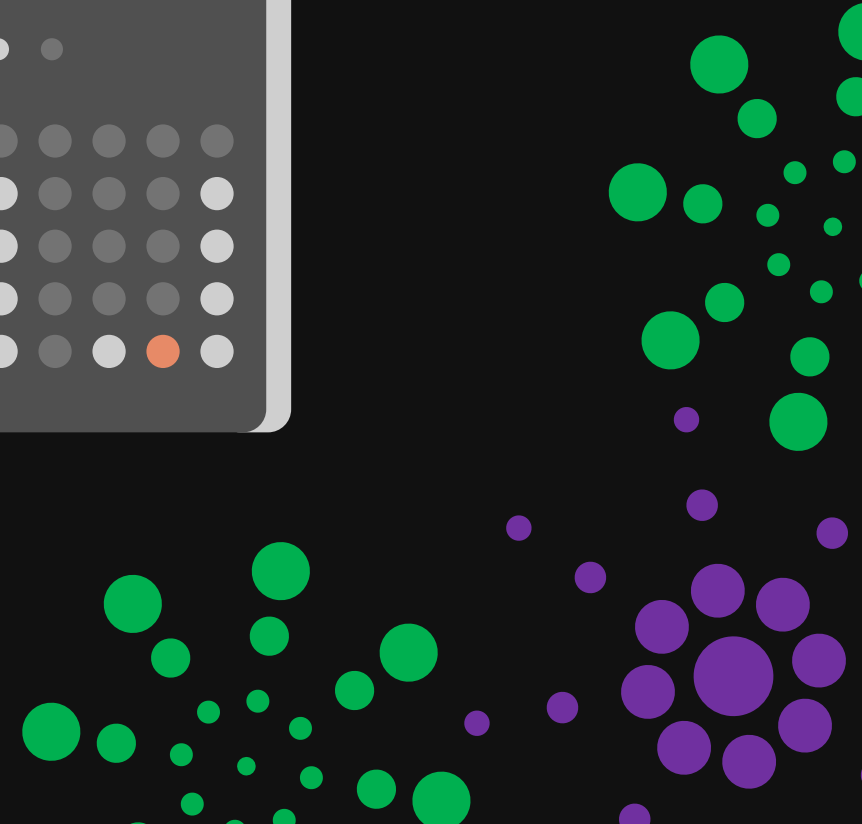
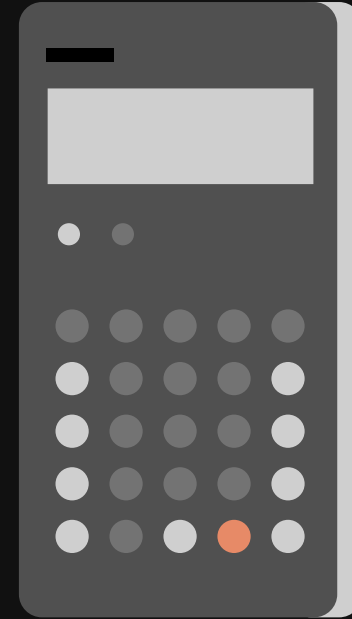
How Frontend Unit Tests Work

- Tests focus on testing individual components or functions in isolation
- Use mocks to allow simulation of parts of the applications that our component depends on



Steps for each test case

- Arrange
 - Setup everything needed
- Act
 - Call the function/action being tested
- Assert
 - Check the result is what you expect



Arrange

```
let calculator
```

```
beforeEach(async() => {  
  calculator = mount(Calculator)  
  const inputButtons = calculator.findAll('button')  
  const clearButton = inputButtons[inputButtons.length - 2]  
  await clearButton.trigger('click')  
})
```

```
test('Accepts Input Successfully: 0', async() => {  
  const inputButton = calculator.findAll('button').find(b => b.text() === '0')  
  await inputButton.trigger('click')  
  
  const calculatorInput = calculator.find('.calculator-input')  
  expect(calculatorInput.text()).toBe('0')  
})
```

Act

```
test('Accepts Input Successfully: 0', async() => {  
  const inputButton = calculator.findAll('button').find(b => b.text() === '0')  
  await inputButton.trigger('click')  
  
  const calculatorInput = calculator.find('.calculator-input')  
  expect(calculatorInput.text()).toBe('0')  
})
```

Assert

```
test('Accepts Input Successfully: 0', async() => {  
  const inputButton = calculator.findAll('button').find(b => b.text() === '0')  
  await inputButton.trigger('click')  
  
  const calculatorInput = calculator.find('.calculator-input')  
  expect(calculatorInput.text()).toBe('0')  
})
```

Arrange

```
let calculator

beforeEach(async() => {
  calculator = mount(Calculator)
  const inputButtons = calculator.findAll('button')
  const clearButton = inputButtons[inputButtons.length - 2]
  await clearButton.trigger('click')

  global.fetch = vi.fn(() =>
    Promise.resolve({
      json: () => Promise.resolve({ result: '8' })
    })
  )
})
```


Arrange

```
test('Displays output Successfully', async() => {  
  const inputButtons = calculator.findAll('button')  
  const inputButton = inputButtons.find(b => b.text() === '4')  
  const operatorButton = calculator.find('.addition-button')  
  const equalButton = calculator.find('.equal-button')  
  await inputButton.trigger('click')  
  await operatorButton.trigger('click')  
  await inputButton.trigger('click')  
  await equalButton.trigger('click')  
  
  await new Promise(resolve => setTimeout(resolve))  
  
  const calculatorOutput = calculator.find('.calculator-output')  
  expect(calculatorOutput.text()).toBe('8')  
})
```

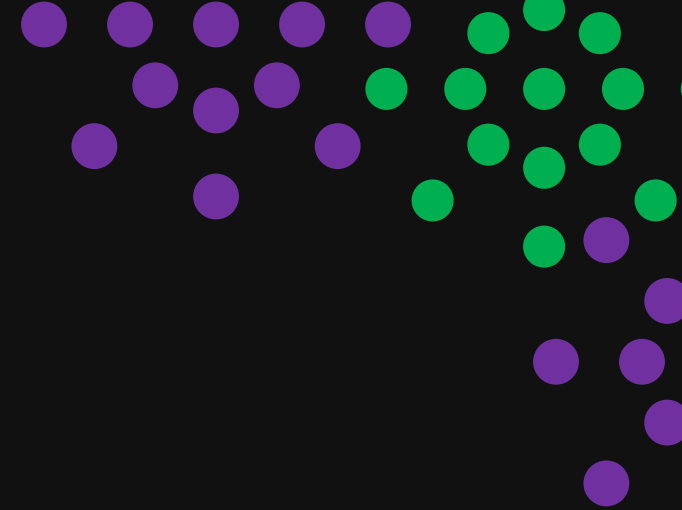
Act

```
test('Displays output Successfully', async() => {  
  const inputButtons = calculator.findAll('button')  
  const inputButton = inputButtons.find(b => b.text() === '4')  
  const operatorButton = calculator.find('.addition-button')  
  const equalButton = calculator.find('.equal-button')  
  await inputButton.trigger('click')  
  await operatorButton.trigger('click')  
  await inputButton.trigger('click')  
  await equalButton.trigger('click')  
  
  await new Promise(resolve => setTimeout(resolve))  
  
  const calculatorOutput = calculator.find('.calculator-output')  
  expect(calculatorOutput.text()).toBe('8')  
})
```

Assert

```
test('Displays output Successfully', async() => {  
  const inputButtons = calculator.findAll('button')  
  const inputButton = inputButtons.find(b => b.text() === '4')  
  const operatorButton = calculator.find('.addition-button')  
  const equalButton = calculator.find('.equal-button')  
  await inputButton.trigger('click')  
  await operatorButton.trigger('click')  
  await inputButton.trigger('click')  
  await equalButton.trigger('click')  
  
  await new Promise(resolve => setTimeout(resolve))  
  
  const calculatorOutput = calculator.find('.calculator-output')  
  expect(calculatorOutput.text()).toBe('8')  
})
```

Write All Unit Tests



```
Test Files  1 passed (1)
  Tests     39 passed (39)
  Start at  21:37:30
  Duration  4.41s (transform 257ms, setup 0ms, import 518ms, test 3.61s)
```



Next Step

Integration Tests